

Lecturer:

Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch)

Teaching Assistant:

Ali Ajorian (ali.ajorian@unibas.ch)

Tutors:

Mark Starzynski (mark.starzynski@unibas.ch)

Sebastian Lenzlinger (sebastian.lenzlinger@unibas.ch)

Uploaded on

Wed., 27. March 2024

Deadline

Sun., 14. April 2024 (23:55)

Upload to ADAM

Modalities

The exercises have to be done in **groups of 2 students**. Upload only the final version of the exercise to Adam and specify your group partner.

Please upload your implementations for suggested API, requested scripts and your short report, containing all answers to the asked questions as a .pdf file to ADAM by the deadline at the latest.

Distributed Offline Chatroom with Git

The goal of this exercise is to familiarize you with offline-first communication in peer-to-peer systems by creating an offline-first chat application. Our application will use a single **public chatroom** model where all users write their messages on a public room. However, since it is a peer-to-peer application, there is no central server for message posting. Instead, we rely on data replication. Each user maintains a replica of all the messages they have received. These messages are stored in append-only linked lists, also known as append-only logs, with a separate log for each message sender. When a user wishes to post a message, they append it to the corresponding log in their local replica. Subsequently, when they are online, the user disseminates their entire replica, including all logs, to a subset of online users. Since all users engage in this behavior, over time, the logs of all users gradually synchronize. Through this process, all users ultimately establish a shared viewpoint of the chatroom, ensuring consistency across the system.

As Git provides local replicas with offline commit mechanism, we can take advantage of its infrastructure to build our application on top of that.

Causal Relationship of Messages: In order to capture the dependencies between messages, it is crucial to consider their causal order. For instance, when a user writes a text to the chatroom and another user replies to it, the second message is causally related to the first message. To maintain this causality across all users messages, each generated message should capture the chatroom's last state. This can be achieved by employing a graph data structure within the messages, where each message includes pointers to the set of the last sent messages from other users, known as frontiers. These pointers indicate that the posted message was created after the user had seen the frontiers set.

Assumptions: All users of the chatroom are trustworthy, meaning that:

- They don't create invalid messages.
- They don't post messages where they shouldn't.
- They don't tamper their replicas.

Design:

- **Repositories as local replicas:** Each replica is modeled as a Git repository.
- **Branches as append-only logs:** In each replica, a separate branch is created for each user from whom a message was received. Each branch is named after the corresponding user, allowing for easy identification and organization of the branches within the replica.
- **Commits as messages:** A message m would be a commit with the comment m and a tree object without any associated blob. So, when someone wants to send a message to the chatroom, they simply add a commit object to their local replica, placing it at the end of the branch that is named after them.
- **Causal Relationship as Parent Pointers:** To capture the causal order of messages, each commit would have all the latest messages from each log as its parent. This captures the most recent viewpoint of the user when they are posting a message.



- **Replication as Pull/Push:** The commands "git pull" and "git push" provide an easy way to disseminate an entire replica to a target user. By providing right arguments to these commands, all messages within a replica, organized in their respective logs, can be efficiently transferred and replicated in another replica, ensuring they are placed in the corresponding logs.

API: You are required to implement the provided function interfaces, which will be invoked by any of the local replicas and collectively shape the entire chatroom. All of the implementations should be carried out using the bash scripting language.

- **Initialize:** This function accepts a variable named 'name' as an input. It establishes a Git repository with the name 'name' and initializes a branch named 'name' and creates an initial commit with the message "*'name' joined the chatroom*".

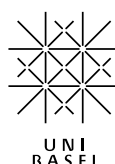
Note: In all of your implementations, the utilization of a blob object is unnecessary. Therefore, you should generate a tree object in the *Initialize* function that does not reference any blob. This tree object should be utilized in all subsequent commit objects that you create.

- **Post:** This function takes a string message as input and posts it in the chatroom under the name of the current repository as the name of sender user. To do this, you need to create a commit with the message content and the correct list of parents must encode the causal relationship between messages. The commit should then be appended at the end of the branch associated with the user who is posting the message.
- **Show:** This function displays the messages in their causal order. The format of each message is "author name (time): message". You can utilize the Git "log" command to generate this output.

Documentation and Implementation: Please make sure to submit all of your implementations, including API functions and scripts, for the requested scenarios. Additionally, include a collection of screenshots that are accompanied by explanations indicating what they depict and to which specific task and subtask they are related.

Task 1 - Initialization (3 points)

- Implement function *Initialize* for our API
- Initialize a replica named Alice using your implementation of the *Initialize* function. Use the command "git cat-file --batch-all-objects --batch-check" to list all the git objects within Alice's replica. Then, use the "git log" command to view the single message created by Alice, which is "*Alice joined the chatroom*".
- Initialize another user named Bob and follow the same steps as before to examine the internal objects and any generated messages. Then, establish a one-way communication channel from Alice to Bob by using the command "git remote add bob ../Bob" in Alice's repository. This setup allows Alice to send all her offline recorded messages to Bob using the "push" command. Make sure to provide the appropriate arguments for



this command to ensure successful communication. Afterward, switch to Bob's repository and list all the Git objects by running the proper command. To view the content of each object, you can use `"git cat-file -p HASH"` and replace "HASH" with the hash of the object you want to inspect.

- (d) In Bob's repository, use the `"git log"` command to see the initial messages from both Alice and Bob. Describe the problem you encounter and provide a solution for it.

Create a document that includes screenshots capturing the results of parts (b), (c), and (d). Provide also a detailed description of the problem faced in part c, as well as its corresponding solution in that document.

Task 2 - Posting Messages to the Chatroom (4 points)

Using your implementation for the function *Post*, post a message *"Hello, everyone! I'm here."* from Alice to the chatroom. Then update Bob's repository and see Alice's message and reply to her, *"Hello, Alice! I'm new here too"*. Now update the Alice's replica and see the disseminated messages.

Using command `"git log --oneline --graph"` in Alice's repository, you should see an output similar to the following picture.

```
* f7d9062 (HEAD, bob/Alice, Alice) Hello, everyone! I'm here.
* bf2c9b5 Alice joined the chatroom
```

(a) *alice* branch

```
* a49fb6b (HEAD -> bob, bob/Bob) Hello, Alice! I'm new here too
|
* 899cf15 Bob joined the chatroom
|
* f7d9062 (bob/Alice, Alice) Hello, everyone! I'm here.
|
* bf2c9b5 Alice joined the chatroom
```

(b) *bob* branch

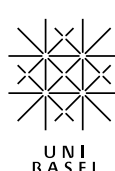
Figure 1: `git log` output for Alice's repository on different branches

Task 3 - User Interface (3 points)

- (a) Implement the function *Show* that demonstrates chatroom messages along with their causal relationship. The function should format the output as 'Author (time): message'.
- (b) Write a script using your implementations for the API that initializes repositories Alice and Bob, establishes a dialogue between them and then prints the result with your function *Show* from a). The resulting output should look similar to Figure 2

```
Alice(19:57 PM):Are you a student?
Alice(19:57 PM):I'm good, thanks for asking.
Bob(19:57 PM):How are you?
Bob(19:57 PM):Hey Alice!
Bob(19:57 PM):Bob joined the chatroom
Alice(19:57 PM):Hey everyone!
Alice(19:57 PM):Alice joined the chatroom
```

Figure 2: A chatroom output instance



- (c) What is the difference between the scenario in which a new user named Carol pulls Alice's repository and posts the message "Hi all" versus the case where Carol posts the message without pulling Alice's repository.

Useful Resources

- Git Internals - Plumbing and Porcelain
- Git docs